

Repair-Optimal Data Placement for Locally Repairable Codes with Optimal Minimum Hamming Distance

Shuang Ma

mashuang2@mail.ustc.edu.cn

University of Science and Technology of China
Hefei, Anhui, China

Cheng Li

University of Science and Technology of China
Hefei, Anhui, China
chengli7@ustc.edu.cn

Si Wu

siwu5938@ustc.edu.cn

University of Science and Technology of China
Hefei, Anhui, China

Yinlong Xu

University of Science and Technology of China
Hefei, Anhui, China
ylxu@ustc.edu.cn

ABSTRACT

Modern clustered storage systems increasingly adopt erasure coding to realize reliable data storage at low storage redundancy. Locally Repairable Codes (LRC) are a family of practical erasure codes with high repair efficiency. Among various LRC constructions, Optimal-LRC is a recently proposed LRC approach that achieves the optimal Minimum Hamming Distance with low theoretical repair costs. In this paper, we consider the repair performance of Optimal-LRC in clustered storage systems. We show that the conventional flat data placement and random data placement incur substantial cross-cluster repair traffic, which impairs the repair performance. To this end, we design an optimal data placement scheme that provably minimizes the cross-cluster repair traffic, by carefully placing each group of blocks in Optimal-LRC into a minimum number of clusters subject to single-cluster fault tolerance. We implement our optimal data placement scheme on a key-value store prototype atop Memcached, and show via LAN testbed experiments that the optimal data placement significantly improves the repair performance compared to the conventional data placements.

CCS CONCEPTS

• **Computer systems organization** → **Redundancy; Reliability**; • **Information systems** → **Storage recovery strategies; Distributed storage**.

KEYWORDS

Erasure coding, Locally Repairable Codes, Data repair, Clustered storage

ACM Reference Format:

Shuang Ma, Si Wu, Cheng Li, and Yinlong Xu. 2022. Repair-Optimal Data Placement for Locally Repairable Codes with Optimal Minimum Hamming Distance. In *51st International Conference on Parallel Processing (ICPP '22)*,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICPP '22, August 29-September 1, 2022, Bordeaux, France

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9733-9/22/08...\$15.00

<https://doi.org/10.1145/3545008.3545038>

August 29-September 1, 2022, Bordeaux, France. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3545008.3545038>

1 INTRODUCTION

In modern large-scale storage systems with hundreds of thousands of storage nodes, failures are the norm rather than the exception [7]. To reliably store data against frequent failures, storage systems increasingly adopt *erasure coding* to provide low-cost redundancy [6, 13, 23]. Compared to replication, erasure coding achieves the same degree of data reliability at much lower redundancy [38]. At a high level, an erasure code encodes k original data blocks into $n - k$ additional *parity blocks*, such that all original data can be reconstructed from a sufficient number of data and parity blocks.

Despite the high storage efficiency, traditional erasure codes such as Reed-Solomon Codes [30] suffer from the repair penalty. For example, the repair of one block in (n, k) Reed-Solomon Codes needs to access k surviving blocks for reconstruction. This greatly amplifies the I/O and network traffic for repair. To mitigate the expensive repair overhead, new erasure codes are proposed. Among them, *Locally Repairable Codes (LRC)* are a family of repair-friendly erasure codes [13, 16, 32]. An LRC can be represented by three parameters (n, k, r) . It partitions k data blocks into short-size groups of size no more than r , and calculates a *local parity block* for each group, such that the repair of one block needs to access only r other blocks in a short-size group. It also computes some *global parity blocks* based on all data blocks for high fault tolerance. Due to the high repair efficiency, LRC is widely deployed in enterprise data centers [13, 16, 32].

There exist a variety of LRC constructions, such as Microsoft's *Azure-LRC* [13], Facebook's *Xorbas* [32], *Azure-LRC+1* developed based on *Azure-LRC* [16], and *Optimal-LRC* [16]. Here, we pay attention to Optimal-LRC, which exhibits several advantages among various LRC constructions. First, Optimal-LRC achieves the *optimal* Minimum Hamming Distance, meaning that it gains the maximum fault tolerance ability with the given parameters [8]. Second, it applies to a wide range of parameters [16]. Third, it has low repair costs by adding local parity blocks for both data blocks and global parity blocks [16]. In spite of such merits, Optimal-LRC is mainly studied theoretically [2, 15, 16, 19, 35, 36]. Fault tolerance and repair performance of Optimal-LRC are unclear in real storage systems.

In this paper, we focus on the repair performance of Optimal-LRC in clustered storage systems, which organize nodes into *clusters*

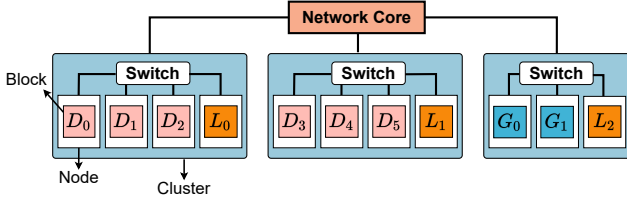


Figure 1: An example of a clustered storage system. A blue colored rectangle indicates a cluster.

such that the cross-cluster network bandwidth is far more scarce than the inner-cluster bandwidth [1, 10, 37]. However, it is challenging to realize optimal repair, as the data placement of the blocks across the clusters greatly affects the repair performance. As we show, the flat data placement and the random data placement amplify the cross-cluster traffic for repair. To this end, we design an *optimal* data placement scheme for Optimal-LRC in clustered storage systems. By carefully determining the placement of each group of blocks across the clusters, our data placement scheme provably minimizes the cross-cluster traffic for repair. Our contributions include.

- We first analyze the fault tolerance of Optimal-LRC, and derive the feasible data placement strategy subject to single-cluster fault tolerance. We then design an optimal data placement scheme for Optimal-LRC that provably minimizes the cross-cluster repair traffic, by placing each group of blocks into a minimum number of clusters under single-cluster fault tolerance. We further derive the repair strategies under the optimal data placements (Section 4).
- We implement a key-value store prototype that realizes Optimal-LRC and the optimal data placement scheme based on Memcached [22]. We conduct extensive experiments on a LAN testbed. Experimental results show that our optimal data placement reduces the degraded read time of the flat data placement and the random data placement by up to 86.2% and 69.2%, respectively, and achieves up to 7.7 $\times$ and 5.6 $\times$ node repair rate gains over the flat data placement and the random data placement, respectively (Section 6).

The source code of our prototype is available at <https://github.com/monopodium/icpp22-opt-lrc.git>.

2 BACKGROUND AND PROBLEM

2.1 Clustered Storage under Optimal-LRC

Clustered storage architecture. Real-world storage systems typically have a layered architecture [4, 10, 33], as shown in Figure 1. The system partitions *nodes* into multiple *clusters*. The nodes within a cluster are interconnected by a switch, while the clusters are further connected by a network core. In such a clustered storage system, the cross-cluster network bandwidth is much more scarce than the inner-cluster bandwidth [1, 37]. For example, Cisco’s research suggests that the ratio of the cross-cluster bandwidth to the inner-cluster bandwidth can be 1:5-1:20 [5]. Thus, the cross-cluster network transfer is the major bottleneck of system operations.

Maximum Distance Separable erasure codes. Modern clustered storage systems increasingly adopt *erasure coding* to provide high data reliability at low storage overhead. An erasure code encodes k original data blocks into $n - k$ additional *parity blocks*, such that it can tolerate the loss of any $t \leq n - k$ blocks. A general metric *Minimum Hamming Distance* (or *distance* for short, denoted by d) is defined as the minimum number of concurrent block failures that would cause data loss [20]. In erasure coding, $d = \max(t + 1)$. If $t = n - k$ (or equivalently, $d = n - k + 1$), then the erasure code is called *Maximum Distance Separable (MDS)* code, which means that it achieves the maximum distance and the maximum fault tolerance ability with the given redundancy. The most popular family of MDS codes are Reed-Solomon codes [30].

The n blocks that are encoded together are called a *stripe*, and are stored on n different nodes to provide node-level fault tolerance. A storage system usually stores multiple stripes, each of which is encoded independently. In this work, we mainly focus on the repair operation of a single stripe.

In an MDS code with parameters n and k , the repair of a lost block requires to access k surviving blocks in the same stripe. This greatly amplifies the I/O and network overhead and impairs the repair performance. For example, a study at Facebook’s warehouse clusters reported that a median of more than 180 TB of data is transferred across clusters every day by Reed-Solomon-coded repair [28]. Hence, storage practitioners try to design new erasure codes with improved repair performance.

Locally Repairable Codes. *Locally Repairable Codes (LRC)* are a family of repair-efficient erasure codes [13, 16, 32]. We construct an LRC with three parameters (n, k, r) . It comprises k data blocks (denoted by $D_0, D_1, \dots, D_{k-1}$), and $n - k$ parity blocks. It divides the data blocks into small-size groups of size no more than r , and adds a *local parity block* (denoted by L_i) for each group, such that the repair of a single lost block can be achieved by accessing the remaining r blocks of the same group. It further generates *global parity blocks* (denoted by $G_0, G_1, \dots$) based on all data blocks for high fault tolerance.

Compared to MDS codes, LRC greatly improves the efficiency of repairing a single block, i.e., an MDS code requires to access k blocks while LRC needs to access only r (r can be much smaller than k) blocks. However, the distance of LRC is not maximized, i.e., $d < n - k + 1$. This means that under the same (n, k) , LRC achieves better repair performance with compromised fault tolerance, compared to MDS codes.

There are various LRC constructions, including Microsoft’s *Azure-LRC* [13], Facebook’s *Xorbas* [32], *Azure-LRC+1* [16], and *Optimal-LRC* [16]. Among them, *Azure-LRC* appears to be the most well-known LRC for its simple construction. However, it optimizes the repair of data blocks only but the repair of global parity blocks is still expensive. *Xorbas*, *Azure-LRC+1*, and *Optimal-LRC* further optimize the repair of global parity blocks by adding local parity blocks for the global parity blocks. However, *Xorbas* is defined for limited parameters and *Azure-LRC+1* has a smaller distance, while *Optimal-LRC* is applicable over a wide range of parameters with a larger distance [16].

We systematically compare the four LRC constructions in Section 6.1 (Figure 8). We find that *Optimal-LRC* shows better repair

performance and adapts to numerous parameters. Despite these merits, Optimal-LRC has not received enough attention by storage practitioners, especially in modern clustered storage. This motivates us to study the repair performance of Optimal-LRC in real clustered storage systems.

Optimal-LRC. An (n, k, r) Optimal-LRC generates global parity blocks from all k data blocks. It then divides all k data blocks and all global parity blocks into groups of size no more than r , and calculates a local parity block for each group. In this manner, the repair of *any* block can be completed by retrieving up to r blocks in the same group. Note that the calculation of the global and local parity blocks is based on Galois Field arithmetics [24]. Optimal-LRC does not require the number of data and global parity blocks to be a multiple of r . However, it requires that $n \bmod (r + 1) \neq 1$. For example, Figure 1 shows an Optimal-LRC stripe with $(n, k, r) = (11, 6, 3)$. It generates two global parity blocks from the six data blocks, e.g., $G_0 = \alpha_0 D_0 + \alpha_1 D_1 + \alpha_2 D_2 + \alpha_3 D_3 + \alpha_4 D_4 + \alpha_5 D_5$, where the coefficients α_i s and arithmetics are over Galois Field. The six data blocks and two global parity blocks are partitioned into three groups. Three local parity blocks are then computed, e.g., $L_0 = \beta_0 D_0 + \beta_1 D_1 + \beta_2 D_2$, and $L_2 = \beta_6 G_0 + \beta_7 G_1$. It can be seen that repairing a data block needs to access three blocks and repairing a global parity block needs two blocks.

Optimal-LRC gains *optimality* in terms of the distance. It has been proved that the upper bound of the distance of an (n, k, r) LRC is $n - k - \lceil \frac{k}{r} \rceil + 2$ [8]. Optimal-LRC can reach this upper bound. In [35], the distance of Optimal-LRC is shown to be

$$d = n - k - \left\lceil \frac{k}{r} \right\rceil + 2, \quad \text{if } (r + 1) \mid n, \quad (1)$$

$$d \geq n - k - \left\lceil \frac{k}{r} \right\rceil + 1, \quad \text{if } (r + 1) \nmid n, r \mid (k + 1). \quad (2)$$

When $(r + 1) \mid n$, the distance of Optimal-LRC is proved to be optimal [35]. For example, the distance of an $(n, k, r) = (12, 6, 3)$ Optimal-LRC is $d = 6$. However, when $(r + 1) \nmid n$, Optimal-LRC is proved to achieve the largest possible distance, but the exact distance is undetermined [15]. In other words, when $(r + 1) \nmid n$, the distance may be $n - k - \lceil \frac{k}{r} \rceil + 1$ or $n - k - \lceil \frac{k}{r} \rceil + 2$. For example, for an $(n, k, r) = (11, 6, 3)$ Optimal-LRC, the distance may be $d = 4$ or $d = 5$. In this paper, we will formally give the concise formula to calculate the distance of Optimal-LRC under $(r + 1) \nmid n$ (Section 4.1).

Fault tolerance of Optimal-LRC. By placing the stripe of n blocks of Optimal-LRC across n nodes, the system provides multi-node fault tolerance. Since cluster failures occur much less frequently than node failures [23, 28], we further enforce the system to provide *single-cluster fault tolerance*. Subject to single-cluster fault tolerance, we can place multiple blocks of a stripe into a single cluster, such that we can greatly reduce the cross-cluster network traffic for repair [41].

2.2 Repair Problem of Optimal-LRC

We focus on the repair operation of a single lost block since single block failures are more common events than multiple block failures in practice [28]. Recall that Optimal-LRC divides all blocks into different groups (e.g., Figure 1). In order to repair an unavailable

block, Optimal-LRC retrieves all other blocks in the same group. In this way, all repair operations for single block failures are completed within a group.

Metrics. In [16], the authors use $Cost(B_i)$ to represent the number of blocks required to repair a block B_i . They further define two metrics to measure the repair performance of Optimal-LRC. The first metric is *degraded read cost* (DRC), which is defined as the average number of blocks required to repair all data blocks, i.e., $DRC = \frac{1}{k} \sum_{i=0}^{k-1} Cost(B_i)$. The second metric is *normalized repair cost* (NRC), which is defined as the number of blocks required to repair all blocks divided by k , i.e., $NRC = \frac{1}{k} \sum_{i=0}^{n-1} Cost(B_i)$. NRC can be reviewed as the cost of repairing a failed node, where the cost of repairing the parity blocks is amortized over the k data blocks [16].

The above two metrics are comprehensive to measure the degraded read performance and the node repair performance of Optimal-LRC. Note that in real storage systems, temporary failures that trigger degraded reads happen frequently, while permanent failures that trigger node repair also happen from time to time [32].

However, the metrics in [16] do not take into account the hierarchical setting of clustered storage. Thus, we extend the metrics to measure the repair performance of Optimal-LRC in clustered storage.

We now define $Cost(B_i)$ as the number of blocks transferred across clusters to repair a block B_i . Then, DRC (i.e., $\frac{1}{k} \sum_{i=0}^{k-1} Cost(B_i)$) represents the average number of blocks transferred across clusters to repair all data blocks, while NRC (i.e., $\frac{1}{k} \sum_{i=0}^{n-1} Cost(B_i)$) indicates the number of blocks transferred across clusters to repair all blocks normalized by k . For example, in Figure 1, $Cost(B_i) = 0$ as the repair of any block can be performed within a single cluster. Thus, $DRC = NRC = 0$. However, as we will show later, the data placement in Figure 1 cannot guarantee single-cluster fault tolerance.

Problem description. Our problem is how to minimize the degraded read cost and normalized repair cost of Optimal-LRC, subject to single-cluster fault tolerance. By minimizing the two costs, we can achieve the optimal repair performance.

3 CHALLENGES AND MOTIVATIONS

3.1 Challenges

High repair costs with flat data placement. Existing studies on LRC adopt the flat data placement, where the n blocks of an LRC stripe are distributed to n different clusters [13, 32, 43]. The flat data placement can guarantee the maximum cluster-level fault tolerance. However, it incurs high repair costs due to the separate storage of the blocks.

For example, Figure 2(a) shows the flat data placement for $(n, k, r) = (12, 6, 3)$ Optimal-LRC. The stripe of 12 blocks is stored in 12 clusters. When any block fails, we need to transfer three blocks across clusters for repair. For example, the repair of D_0 needs to transfer D_1 , D_2 , and L_0 to the cluster where D_0 is stored. Thus, $Cost(B_i) = 3$ for any block B_i . We can further calculate that $DRC = 3$ and $NRC = 6$.

Sub-optimal repair performance with random data placement. The reason why the flat data placement suffers from high repair costs is

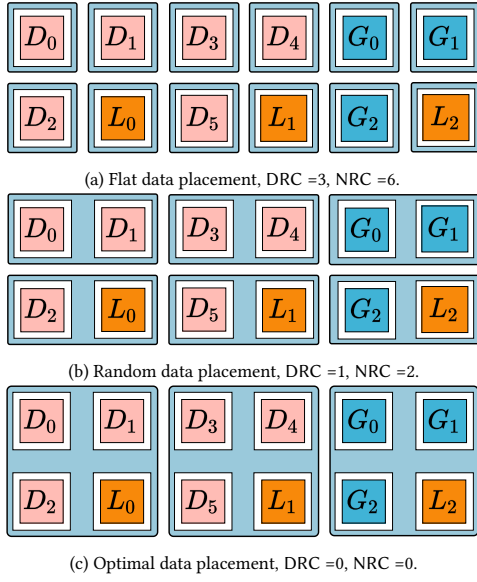


Figure 2: Different data placements of (12,6,3) Optimal-LRC.

that it stores the blocks in distinct clusters to maintain the maximum cluster-level fault tolerance. We can place multiple blocks of a stripe into a single cluster to maintain single-cluster fault tolerance. We then study the random data placement subject to single-cluster fault tolerance [16, 39, 40, 44].

For example, Figure 2(b) shows a random data placement that guarantees single-cluster fault tolerance for (12, 6, 3) Optimal-LRC. Each group of four blocks is stored in two clusters (e.g., D_0, D_1, D_2 , and L_0 span two clusters). For the repair of any block, we apply *encode-and-transfer* [10, 33, 41] to the two clusters that store the corresponding group of blocks. For example, if D_0 fails, we encode a partial block (i.e., $L_0 + \beta_2 D_2$) from D_2 and L_0 at the cluster that stores D_2 and L_0 , and transfer the partial block to the cluster that stores D_0 . We retrieve D_1 at the same cluster as D_0 , and reconstruct D_0 via $(L_0 + \beta_2 D_2 + \beta_1 D_1)/\beta_0$. Thus, $Cost(B_i) = 1$ for any block B_i . We further infer that $DRC = 1$ and $NRC = 2$.

Random data placement shows notable saving over flat data placement, yet it still cannot promise optimality in terms of the repair costs. Besides, random data placement cannot guarantee stable repair performance. For example, if the number of clusters in the system is large, then the random data placement may incur significant repair costs.

Inefficiency of extending data placement for Azure-LRC to Optimal-LRC. Recent studies propose data placement that minimizes the degraded read cost for Azure-LRC [10, 41]. However, we cannot extend the data placement for Azure-LRC to Optimal-LRC, mainly because the two LRC constructions have different fault tolerance abilities and locality semantics. For example, Azure-LRC collocates every $n - k - \lceil k/r \rceil + 1$ blocks in one cluster as it tolerates any $n - k - \lceil k/r \rceil + 1$ block failures. However, if we colocate $n - k - \lceil k/r \rceil + 1$ blocks in one cluster for Optimal-LRC, it may break single-cluster fault tolerance (e.g., for (11, 6, 3) Optimal-LRC, it violates fault tolerance to place four blocks in one cluster).

3.2 Motivation

Minimize the number of clusters spanned by each group of blocks. Figure 2(b) implies that if the number of clusters each group of blocks spans is reduced, then the repair costs are reduced. Our idea is to minimize the number of clusters spanned by each group of blocks subject to single-cluster fault tolerance. By doing this, we can minimize the repair costs, and achieve the optimal repair performance.

For example, Figure 2(c) shows a third data placement for (12, 6, 3) Optimal-LRC. Each group of four blocks is placed in one cluster. The repair of any block can be achieved within a single cluster, implying that $Cost(B_i) = 0$, $DRC = 0$, and $NRC = 0$. Thus, the repair costs are minimized. We will also prove that the data placement of Figure 2(c) keeps single-cluster fault tolerance (Section 4.1).

Note that we cannot simply put each group of blocks into one cluster for Optimal-LRC with general parameters. For example, for (11, 6, 3) Optimal-LRC, it violates single-cluster fault tolerance to distribute each group of blocks to one separate cluster. Hence, we need to derive the fault tolerance ability of Optimal-LRC, and then design the optimal data placement.

4 OPTIMIZE DATA PLACEMENT FOR REPAIR

We design an optimal data placement scheme for repair in Optimal-LRC. The core idea is to minimize the number of clusters spanned by each group of blocks subject to single-cluster fault tolerance. To achieve this, we first explore the data placement strategy subject to single-cluster fault tolerance in Optimal-LRC (Section 4.1). We then present our optimal data placement scheme (Section 4.2), and the repair policies under the optimal data placements (Section 4.3). We finally discuss how to distribute multiple stripes among multiple clusters to achieve load balance (Section 4.4).

4.1 Fault tolerance of Optimal-LRC

We denote the number of local parity blocks and the number of global parity blocks as l and g , which can be readily calculated from n , k , and r . Specifically, $l = \lceil \frac{n}{r+1} \rceil$ and $g = n - k - l = n - k - \lceil \frac{n}{r+1} \rceil$.

We analyze the number of block failures tolerated by Optimal-LRC, based on which we can derive the distance d (i.e., the minimum number of block failures that would cause data loss).

LEMMA 1. An (n, k, r) Optimal-LRC can tolerate arbitrary $g + \lceil \frac{g+1}{r} \rceil$ block failures.

PROOF. Note that when $g \bmod r \neq 0$, $g + \lceil \frac{g+1}{r} \rceil = g + \lceil \frac{g}{r} \rceil$. When $g \bmod r = 0$, $g + \lceil \frac{g+1}{r} \rceil = g + \frac{g}{r} + 1$.

We first assume that $g \bmod r \neq 0$. Consider a set of up to $g + \lceil \frac{g}{r} \rceil$ failed blocks. Since there are $r + 1$ blocks in each group, the failed blocks span at least $\lceil (g + \lceil \frac{g}{r} \rceil) / (r + 1) \rceil = \lceil \frac{g}{r} \rceil$ groups. There are $\lceil \frac{g}{r} \rceil$ associated local parity blocks and g global parity blocks that can be used for decoding. Thus, the set of $g + \lceil \frac{g}{r} \rceil$ failed blocks can be successfully repaired [13, 41]. However, if we have $g + \lceil \frac{g}{r} \rceil + 1$ failed blocks, then they cannot be successfully repaired as there are only $g + \lceil \frac{g}{r} \rceil$ relevant parity blocks.

We next consider the cases where $g \bmod r = 0$. We consider a set of up to $g + \frac{g}{r} + 1$ failed blocks. These failed blocks span at least $\frac{g}{r} + 1$ groups. The number of failed blocks, i.e., $g + \frac{g}{r} + 1$, is

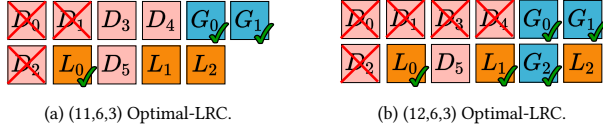
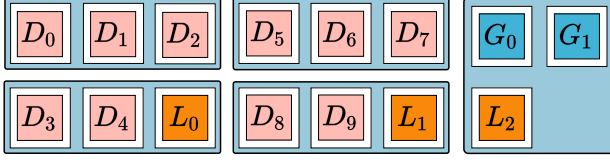


Figure 3: Fault tolerance illustrations in Optimal-LRC

Figure 4: Optimal data placement for $(n, k, r) = (15, 10, 5)$ Optimal-LRC.

equal to the sum of the number of associated local parity blocks (i.e., $\frac{g}{r} + 1$) and the number of global parity blocks (i.e., g). Then, these block failures can be tolerated. However, if we have $g + \frac{g}{r} + 2$ block failures, then the fault tolerance may fail, as now the number of relevant parity blocks (i.e., $g + \frac{g}{r} + 1$) is smaller than the number of failed blocks (i.e., $g + \frac{g}{r} + 2$). $\square$

By Lemma 1, we can infer that we have to place no more than $g + \lceil \frac{g+1}{r} \rceil$ blocks in one cluster to maintain single-cluster fault tolerance for an (n, k, r) Optimal-LRC.

Besides, Lemma 1 also implies that the distance of an (n, k, r) Optimal-LRC is

$$d = g + \left\lceil \frac{g+1}{r} \right\rceil + 1. \quad (3)$$

Note that Equation (3) is consistent with Equation (1) and Equation (2).

For example, for $(11, 6, 3)$ Optimal-LRC in Figure 3(a), we can calculate that $d = 4$ and it tolerates any three block failures. For the three failed blocks D_0 , D_1 , and D_2 depicted, we can utilize L_0 , G_0 , and G_1 for decoding. It also means that we can place three blocks in one cluster to maintain single-cluster fault tolerance. Thus, the data placement in Figure 1 violates single-cluster fault tolerance.

For $(12, 6, 3)$ Optimal-LRC in Figure 3(b), $d = 6$ and any five-block failures can be tolerated. For the five failed blocks D_0 - D_4 depicted, we can use L_0 - L_1 and G_0 - G_2 for efficient decoding. We can also deduce that the data placement in Figure 2(c) keeps single-cluster fault tolerance.

4.2 Data Placement Design.

Overall idea. Based on the fault tolerance analysis in Section 4.1, we can place no more than $d - 1$ blocks into one cluster to promise single-cluster fault tolerance in Optimal-LRC. Therefore, to achieve the minimum DRC and NRC, we place every $d - 1$ blocks of each group in one separate cluster, so as to minimize the number of clusters spanned by each group under single-cluster fault tolerance.

Algorithm details. We now present a data placement algorithm (Algorithm 1) that is guaranteed to minimize DRC and NRC. It

Algorithm 1: Optimal data placement for repair

Input: (n, k, r) for Optimal-LRC
Output: A data placement with minimum repair costs

- 1 **Calculate** l as $\lceil \frac{n}{r+1} \rceil$;
- 2 **Calculate** g as $n - k - \lceil \frac{n}{r+1} \rceil$;
- 3 **Calculate** d as $g + \lceil \frac{g+1}{r} \rceil + 1$;
- 4 **for the** i -th ($0 \leq i < l$) **group do**
- 5 **if** number of blocks in i -th group $\leq d - 1$ **then**
- 6 // This group spans one cluster;
- 7 Select a new cluster;
- 8 Put the i -th group into this cluster;
- 9 **else**
- 10 // This group spans more than one cluster;
- 11 Divide this group into small sets of size $d - 1$;
- 12 **for each set of** $d - 1$ **blocks do**
- 13 Select a new cluster;
- 14 Put the $d - 1$ blocks into this cluster;
- 15 **end**
- 16 **end**
- 17 **end**

takes the three parameters (n, k, r) of an Optimal-LRC as input, and generates a deterministic data placement as output.

We first calculate l , g , and the distance d (Lines 1-3). We then determine the placement for each group of blocks. If the number of blocks in a group is no more than $d - 1$, then this group can be entirely put into one cluster. We thus select a new cluster and place this group of blocks into this cluster (Lines 7-8). If the number of blocks in a group is larger than $d - 1$, then this group has to span two or more clusters to keep single-cluster fault tolerance. We divide this group into small sets of blocks with sizes no more than $d - 1$ (Line 11). For each set of $d - 1$ blocks in this group, we select a new cluster and place the $d - 1$ blocks into this cluster (Lines 13-14).

Example. Figure 4 shows the designed data placement for $(15, 10, 5)$ Optimal-LRC. We can derive that $l = 3$, $g = 2$, and $d = 4$. As the first group consists of six blocks and we can place up to $d - 1 = 3$ blocks in one cluster, we place D_0 - D_2 in one cluster and D_3 - D_4 and L_0 in a different cluster. Similarly, for the second group, we place D_5 - D_7 , and D_8 - D_9 and L_1 in two different clusters. The third group comprises three blocks (i.e., G_0, G_1, L_2), so we put it entirely in one different cluster.

By placing every $d - 1$ blocks into one cluster, we place each of the first two groups into two clusters and the third group into one cluster. Thus, the cost for repairing any block in the first two groups is one cross-cluster block (by applying encode-and-transfer), while the cost for repairing any block in the third group is zero. We can further calculate that DRC = 1 and NRC = 1.2.

Optimality guarantee. We can derive the minimized cost for repairing any block by the following Theorem.

THEOREM 1. For any data placement of an (n, k, r) Optimal-LRC that maintains single-cluster fault tolerance, the minimum cost for repairing any block in a group of size s is $\left\lceil \frac{s}{d-1} \right\rceil - 1$.

Algorithm 2: Single block repair under optimal data placement**Input:** Failed block B_i **Output:** Repair policy for B_i

```

1 Find the group of  $B_i$ ;
2 Find the decoding cluster and helper clusters;
3 if number of blocks in  $B_i$ 's group  $\leq d - 1$  then
4   // Conduct repair in decoding cluster;
5   Read  $B_0-B_{i-1}$ ,  $B_{i+1}-B_r$  and decode  $B_i$ ;
6 else
7   // Apply encode-and-transfer in multiple clusters;
8   for each helper cluster do
9     Read blocks in same group and calculate a partial
       block;
10    Transfer partial block to decoding cluster;
11  end
12  Read other blocks in same group in decoding cluster;
13  Collect all partial blocks and decode  $B_i$ ;
14 end

```

PROOF. By Lemma 1, we can place up to $d - 1$ blocks into one cluster to maintain single-cluster fault tolerance. By placing every $d - 1$ blocks of a group of size s into one separate cluster, we distribute the group into $\lceil \frac{s}{d-1} \rceil$ clusters. The cost for repairing any block is thus $\lceil \frac{s}{d-1} \rceil - 1$.

Suppose there exists a data placement that reduces the cost for repairing a block in a group of size s to $\lceil \frac{s}{d-1} \rceil - 2$. In this case, the group spans $\lceil \frac{s}{d-1} \rceil - 1$ clusters and each cluster stores an average of $s / (\lceil \frac{s}{d-1} \rceil - 1) > d - 1$ blocks. By the Pigeonhole principle [31], there exists a cluster that stores more than $d - 1$ blocks. This breaks single-cluster fault tolerance. $\square$

For simplicity, we can assume that $(r + 1) \mid n$. Then, the size of each group is exactly $r + 1$. From Theorem 1, the minimum cost for repairing any block is $\lceil \frac{r+1}{d-1} \rceil - 1$. We can further calculate the minimum DRC = $\lceil \frac{r+1}{d-1} \rceil - 1$ and the minimum NRC = $\frac{n}{k} \times (\lceil \frac{r+1}{d-1} \rceil - 1)$ (assuming that $(r + 1) \mid n$). Algorithm 1 achieves the minimum cost for repairing any block by collocating up to $d - 1$ blocks of a group in one cluster. Hence, it minimizes the repair costs DRC and NRC.

Computational cost analysis. In Algorithm 1, the placement of each group is readily computed by referring to the number of blocks in each group, so Algorithm 1 brings in negligible computational cost (compared to network transmission cost).

4.3 Repair Policies.

Overall idea. Note that our focus is the repair of a single lost block, which can be achieved by retrieving other blocks within the same group. Under the optimal data placement designed in Section 4.2, each group spans one or more clusters depending on the number of blocks in this group. If a group spans one cluster, we then conduct repair within one cluster. Otherwise, we apply encode-and-transfer to the clusters spanned by this group. By doing this, we can achieve the minimized cross-cluster network traffic for repairing any block, and the minimum repair costs DRC and NRC.

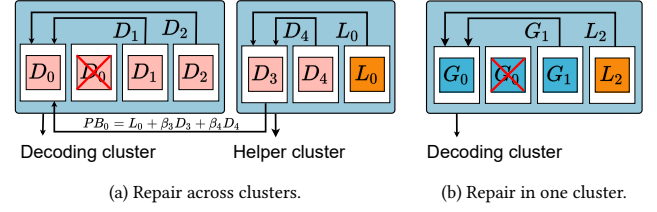


Figure 5: Repair of Optimal-LRC.

Algorithm details. We denote the blocks in a group as $B_0, B_1, \dots, B_r$ without distinguishing data blocks from parity blocks. Let B_i be the failed block. We call the cluster that stores B_i the *decoding cluster*, and a cluster (different from the decoding cluster) that stores other blocks in the same group a *helper cluster*. For example, in Figure 4, if D_0 fails, then the cluster that stores D_0 is the decoding cluster, and the cluster that stores D_3, D_4 , and L_0 is a helper cluster.

Algorithm 2 shows the repair algorithm, which determines the repair policy for a failed block B_i . We first find the group of blocks $B_0, B_1, \dots, B_r$ (Line 1), and the decoding cluster and helper clusters (Line 2). If the group spans one cluster, we then retrieve B_0-B_{i-1} and $B_{i+1}-B_r$ in the decoding cluster, and decode B_i (Line 5). If the group spans two or more clusters, we then apply encode-and-transfer to the clusters. In each helper cluster, we retrieve all blocks in the same group, calculate a partial block, and transfer the partial block to the decoding cluster (Lines 8-11). In the decoding cluster, we first retrieve other blocks in the same group (Line 12), then collect all partial blocks and decode B_i (Line 13).

Examples. Figure 5(a) illustrates the repair process of D_0 in (15, 10, 5) Optimal-LRC. D_0 's group spans two clusters, so we conduct repair across clusters. In the helper cluster, we read D_3, D_4 , and L_0 , calculate a partial block $PB_0 = \beta_3 D_3 + \beta_4 D_4 + L_0$, and transfer PB_0 to the decoding cluster. In the decoding cluster, we read D_1, D_2 , collect PB_0 , and decode D_0 via $(\beta_1 D_1 + \beta_2 D_2 + PB_0) / \beta_0$.

Figure 5(b) illustrates the repair process of G_0 . G_0 's group spans one cluster, so we conduct repair in one cluster. In the decoding cluster, we read G_1, L_2 , and decode G_0 via $(\beta_{11} G_1 + L_2) / \beta_{10}$.

Overhead analysis. Since the cross-cluster network transfer is the bottleneck, the main overhead of Algorithm 2 is the transfer of partial blocks across clusters (Line 10). As we apply encode-and-transfer based on the optimal data placement, we can minimize the cross-cluster overhead for repair.

4.4 Load Balance.

For a single stripe, we use Algorithm 1 to allocate the blocks to some nodes in some clusters. However, the clusters and nodes are randomly selected from the system. For multiple stripes, we expect that the random selection of the clusters and nodes achieves cluster-level and node-level load balance.

5 SYSTEM DESIGN AND IMPLEMENTATION

We prototype a distributed key-value store that realizes Optimal-LRC and the optimal data placement scheme. We present the architecture of the prototype (Section 5.1), the metadata management

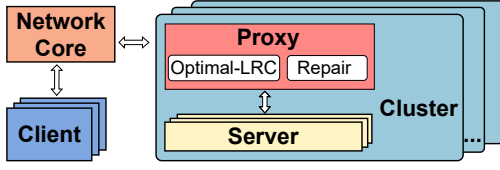


Figure 6: Architecture of our key-value store prototype.

of the prototype (Section 5.2), followed by our implementation on Libmemcached [18] (Section 5.3).

5.1 Architecture

Note that erasure coding is implemented in numerous key-value stores to provide high reliability for hot user data [10, 27]. We also implement Optimal-LRC as a key-value store to show the repair benefits brought by Optimal-LRC for hot data.

Figure 6 shows the architecture of our storage prototype, which consists of multiple clients, and multiple clusters connected by a network core. Each cluster comprises a proxy and several servers. The client is responsible for interacting with user applications. The servers are in charge of storing the data, and the proxy in each cluster provides an interface for the clients to access the storage data. The proxy implements the encoding strategy of Optimal-LRC and the optimal data placement scheme. It realizes basic I/O operations (e.g., PUT, GET, DELETE, etc.), and the degraded read and node repair processes. In particular, the repair of a failed block is coordinated by the proxy. Specifically, the proxies in the helper clusters download relevant blocks, compute partial blocks, and transfer partial blocks to the decoding cluster. The proxy in the decoding cluster downloads other blocks in the same group, collects all partial blocks, and reconstructs the lost block.

5.2 Metadata Management

When applying erasure coding to objects in key-value stores, there are two approaches, *per-object coding* [27] and *cross-object coding* [3]. Per-object coding performs encoding on each object, while cross-object coding combines multiple objects into blocks and collects blocks for encoding. Since per-object coding exhibits better I/O performance and load balance [27], we adopt per-object coding in our implementation.

Specifically, the proxies are responsible for the per-object coding on Optimal-LRC. For each object, one proxy will divide and encode it into k data blocks and $n - k$ parity blocks, both of which are partitioned into different groups. Each of the n blocks is stored as a new key-value pair, and the metadata (e.g., key length, value length) of each new key-value pair is maintained by each server. For each block, the proxy maintains the group information of the block, the cluster ID of the block, and the server ID of the block. When repair operation happens, the proxy searches the metadata and retrieves other blocks in the same group for efficient repair.

5.3 Implementation

We implement our prototype on top on Libmemcached (v1.0.18) [18], by adding about 3000 SLoC. We deploy the Libmemcached as the proxy. We also deploy multiple Memcached servers [22] as

storage servers. We leverage the Jerasure Library [25] for erasure coding. To show the improvements of the optimal data placements over flat and random data placements, we also implement the flat and random data placements.

6 EVALUATION

We evaluate our repair-optimal data placement (denoted by *R-Opt*) via numerical analysis and testbed experiments. We compare R-Opt with the flat data placement (denoted by *Flat*) which stores each block of a stripe in a distinct cluster [13, 32, 43], and the random data placement (denoted by *Random*) which stores the blocks at random subject to single-cluster fault tolerance [16, 39, 40, 44].

The overall results are summarized as follows. First, from numerical analysis, R-Opt greatly reduces the cross-cluster repair traffic of Flat and Random, and completely eliminates the cross-cluster repair traffic under certain parameters (Section 6.1). Second, from testbed experiments, R-Opt reduces the degraded read time of Flat and Random by up to 86.2% and 69.2%, respectively, and achieves up to 7.7 $\times$ and 5.6 $\times$ node repair rate gains over Flat and Random, respectively (Section 6.2).

6.1 Numerical Analysis

Our numerical analysis considers two metrics, i.e., DRC and NRC. We also compare Optimal-LRC with other LRC constructions to show its advantages in clustered storage.

Flat. In Flat, the repair of any failed block requires to transfer across clusters all other blocks in the same group. For simplicity, we assume n is divisible by $r + 1$. Thus, each group has exactly $r + 1$ blocks, and repairing any block requires r cross-cluster blocks, i.e., $Cost(B_i) = r$. We further compute that $DRC = r$ and $NRC = \frac{n}{k} \times r$.

Random. We develop a simulation program to generate ten random data placements for each set of parameters (n, k, r) . We then calculate the average DRC and NRC of the ten random data placements as the repair costs of Random.

R-Opt. In R-Opt, we place every $d - 1$ blocks of a group into a separate cluster, so each group spans $\lceil \frac{r+1}{d-1} \rceil$ clusters. The repair of any block needs $\lceil \frac{r+1}{d-1} \rceil - 1$ cross-cluster blocks, i.e., $Cost(B_i) = \lceil \frac{r+1}{d-1} \rceil - 1$. Further, $DRC = \lceil \frac{r+1}{d-1} \rceil - 1$ and $NRC = \frac{n}{k} \times (\lceil \frac{r+1}{d-1} \rceil - 1)$.

Comparisons of the three data placements. We consider the parameters under $6 \leq n \leq 24, 3 \leq k \leq 15, 2 \leq r \leq 7$, and the redundancy ratio (i.e., $\frac{n}{k}$) from 1.3 to 2.67. We consider the common parameters used in enterprise storage systems, for example $(n = 16, k = 12, r = 6)$ (used in Microsoft Azure [13]) and $(n = 16, k = 10, r = 5)$ (used in Facebook HDFS [32]). Figure 7 shows the repair costs of Flat, Random, and R-Opt under 25 sets of coding parameters. We summarize the following observations.

- R-Opt always has the minimum DRC and NRC. R-Opt reduces the DRC and NRC of Flat significantly, and also shows notable improvements over Random. For example, for $(12, 6, 3)$ Optimal-LRC, $DRC = 3$ and $NRC = 6$ in Flat, $DRC = 1.5$ and $NRC = 4.0$ in Random, and $DRC = 0$ and $NRC = 0$ in R-Opt.
- For the cases where $d - 1 \geq r + 1$, R-Opt places each group of blocks in one cluster, so the cost for repairing any block is

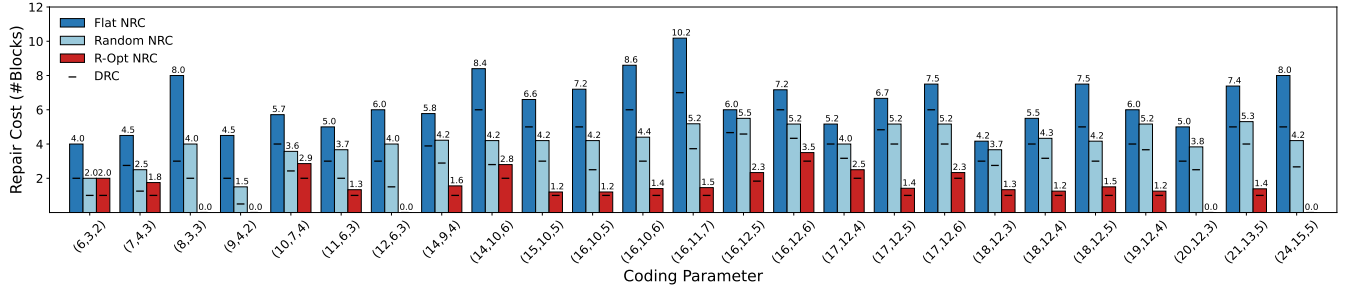


Figure 7: Repair costs of three data placements.

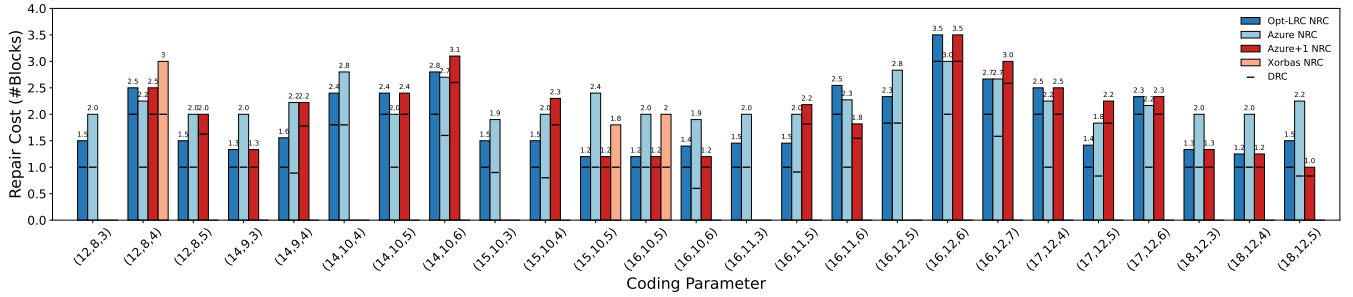


Figure 8: Repair costs of four LRC constructions.

zero, implying that $DRC = NRC = 0$. For example, for Optimal-LRC with parameters $(8, 3, 3)$, $(9, 4, 2)$, $(12, 6, 3)$, $(20, 12, 3)$, $(24, 15, 5)$, $DRC = NRC = 0$.

- As r increases, DRC and NRC increase in Flat, while they keep stable in R-Opt. For example, for $(16, 10, 5)$ Optimal-LRC, $DRC = 5$ and $NRC = 7.2$ in Flat, and $DRC = 1.0$ and $NRC = 1.2$ in R-Opt, while for $(16, 10, 6)$ Optimal-LRC, $DRC = 6$ and $NRC = 8.6$ in Flat, and $DRC = 1.0$ and $NRC = 1.4$ in R-Opt.
- NRC is always larger than DRC because the cost for repairing the parity blocks is amortized over the data blocks.

Comparisons of four LRC constructions. We now compare Optimal-LRC with Azure-LRC [13], Xorbas [32], and Azure-LRC+1 [16]. For Optimal-LRC, we adopt our R-Opt data placement. For the three other LRCs, we derive the data placements in clustered storage via three steps. i) Calculate the distance d . ii) For the groups that contain data blocks, partition every $d - 1$ blocks in a separate cluster. iii) Place the group that contains global parity blocks into a different cluster. These data placements also optimize the repair costs of the three LRCs and meanwhile maintain single-cluster fault tolerance.

We select a series of parameters to compare the repair costs of the four LRCs. Figure 8 shows the results. We have the following observations, which can explain the merits of Optimal-LRC over other LRCs.

- Optimal-LRC and Azure-LRC apply to all parameters, while Azure-LRC+1 and Xorbas apply to limited parameters. In particular, there are only three sets of parameters for Xorbas in Figure 8.

- Since the distance of Azure-LRC+1 is smaller than Azure-LRC and Optimal-LRC by one for most cases, we can put fewer blocks in one cluster, and the number of clusters spanned by a group is larger in Azure-LRC+1. Thus, in most cases, the repair costs of Azure-LRC+1 are larger than that of Azure-LRC and Optimal-LRC.
- In most cases, Optimal-LRC has smaller NRC than Azure-LRC, as the global parity blocks of Optimal-LRC can be repaired locally. Since Optimal-LRC and Azure-LRC have similar distances, they have similar DRC for most cases. For example, for $(12, 8, 3)$, $DRC = 1$ and $NRC = 1.5$ in Optimal-LRC, while $DRC = 1$ and $NRC = 2$ in Azure-LRC.

6.2 Testbed Experiments

Setup. We deploy our key-value store prototype on a local cluster which consists of nine physical nodes, each of which runs Ubuntu 16.04.5 LTS with a quad-core 3.40 GHz Intel Core i5-3570, 16 GB RAM, and a Seagate ST1000DM003 7200 RPM 1 TB SATA hard disk. The nodes are connected by a 10 Gbps network (measured by iperf). We assign one node as the network core, and use each of the remaining eight nodes to emulate a cluster. In each cluster, we deploy one proxy and multiple servers. We use the Wonder Shaper tool [14] to control the outgoing bandwidth of the network core to make the cross-cluster bandwidth constrained.

Methodology. We assume the following default configurations. We adopt $(n, k, r) = (12, 6, 3)$ Optimal-LRC. We consider various object sizes from 1 KB to 4 MB. We configure the bandwidth of the network core to be 1 Gbps (i.e., the ratio of the cross-cluster bandwidth to the inner-cluster bandwidth is 1:10). We vary some

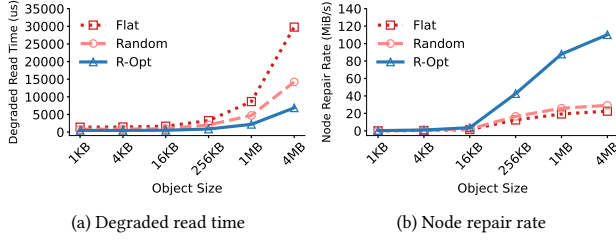


Figure 9: Exp#1: Repair performance under different object sizes.

of the settings in our experiments. We measure the *degraded read time*, which is the average time to repair all data blocks in a stripe. We further upload several objects into the system to let the total storage volume be around 1 GB. We then measure the *node repair rate*, which is the average rate to repair all nodes in a system. Note that node repair involves the repair of data and parity blocks. The results of each experiment are averaged over ten runs.

Experiment 1 (Repair performance under different object sizes). We first evaluate the repair performance of R-Opt under different object sizes, varied from 1 KB to 4 MB. We consider (12, 6, 3) Optimal-LRC, and adopt the default bandwidth configuration. Figure 9 shows the results.

From Figure 9(a), the degraded read time increases with a larger object size, and R-Opt constantly outperforms Flat and Random. According to the theoretics, R-Opt has a much smaller DRC than Flat and Random. Thus, the experimental results comply with the theoretics. Overall, R-Opt reduces the degraded read time of Flat and Random by 65.2% and 40.0%, 66.9% and 46.3%, 68.8% and 44.6%, 73.4% and 57.5%, 74.7% and 53.5%, 76.9% and 51.7%, respectively, across all object sizes. We can see that R-Opt achieves larger gains with a larger object size, as the cost for transferring blocks across clusters increases with a larger object size.

From Figure 9(b), the node repair rate also increases with a larger object size, while R-Opt achieves larger node repair rates. Overall, R-Opt achieves 2.4× and 1.6×, 2.5× and 1.6×, 2.5× and 1.8×, 3.4× and 2.6×, 4.6× and 3.4×, 4.9× and 3.8× node repair rate gains over Flat and Random, respectively, across all object sizes. R-Opt also shows larger gains with a larger object size.

Experiment 2 (Repair performance under different coding parameters). We next evaluate the repair performance under different coding parameters. We consider five sets of (n, k, r) , i.e., (8, 3, 3), (12, 6, 3), (15, 10, 5), (18, 12, 5), and (24, 15, 5). We consider two object sizes, 1 MB and 4 MB. Figure 10 shows the results (the error bars show the maximum and minimum results across ten runs).

From Figure 10(a), the degraded read time decreases with larger parameters. The reason is that we adopt per-object coding, and larger parameters lead to smaller-size blocks. Overall, R-Opt reduces the degraded read time of Flat and Random by 72.2% and 54.9%, 74.7% and 53.5%, 62.8% and 32.5%, 60.7% and 41.8%, and 83.3% and 68.9%, under (8, 3, 3), (12, 6, 3), (15, 10, 5), (18, 12, 5), and (24, 15, 5), respectively. From Figure 10(b), R-Opt achieves 4.5× and 2.8×, 4.6× and 3.4×, 2.4× and 1.4×, 2.4× and 1.6×, 5.9× and 2.7× node repair rate gains over Flat and Random, under the five sets of parameters,

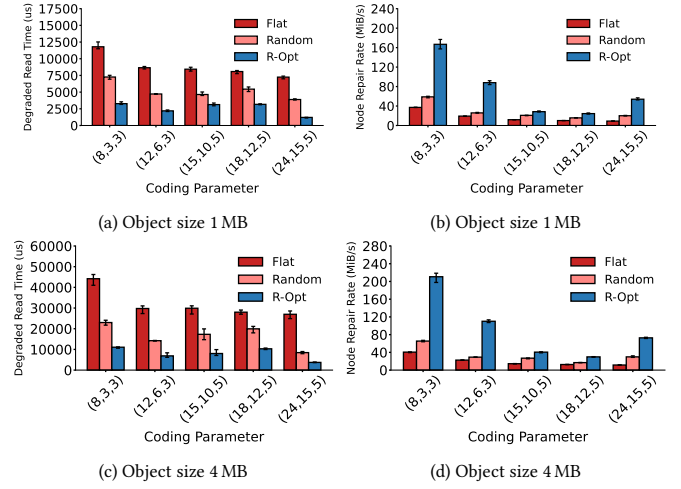


Figure 10: Exp#2: Repair performance under different coding parameters.

respectively. This validates the applicability of R-Opt over different parameters.

From Figure 10(c), R-Opt reduces the degraded read time of Flat and Random by 75.0% and 51.9%, 76.9% and 51.7%, 73.2% and 53.6%, 63.2% and 48.4%, and 86.2% and 56.0%, under the five sets of parameters, respectively. From Figure 10(d), R-Opt achieves 5.2× and 3.2×, 4.9× and 3.8×, 2.8× and 1.5×, 2.4× and 1.8×, and 6.3× and 2.4× node repair rate gains over Flat and Random, under the five sets of parameters, respectively.

Experiment 3 (Repair performance under different cross-cluster bandwidths). We now evaluate the repair performance under different cross-cluster bandwidths. We adopt (12, 6, 3) Optimal-LRC and fix the object size as 4 MB. We vary the cross-cluster bandwidth from 0.5 Gbps to 10 Gbps (i.e., the ratio of inner-cluster bandwidth to cross-cluster bandwidth is 20:1-1:1). Figure 11 shows the results.

From Figure 11(a), the degraded read time decreases with a larger bandwidth. Besides, R-Opt reduces the degraded read time of Flat and Random by 85.5% and 69.2%, 82.4% and 61.6%, 76.9% and 51.7%, 66.7% and 40.3%, 44.6% and 28.8%, when the bandwidth is 0.5 Gbps, 0.7 Gbps, 1 Gbps, 2 Gbps, 10 Gbps, respectively. This indicates that R-Opt achieves larger gains under a more scarce cross-cluster bandwidth.

From Figure 11(b), the node repair rate increases with a larger bandwidth. Overall, R-Opt achieves 7.7× and 5.6×, 6.4× and 4.6×, 4.9× and 3.8×, 3.2× and 2.6×, and 1.9× and 1.8× node repair rate gains over Flat and Random, under the five bandwidth settings, respectively.

Experiment 4 (Load balance). In our experiments, we record the amounts of data in each node and in each cluster. At the node level, the ratio of the amount of data in each node to the average amount of data in all nodes ranges from 0.93 to 1.14. At the cluster level, the ratio of the amount of data in each cluster to the average amount of data in all clusters ranges from 0.98 to 1.05. We can see that R-Opt can balance the storage at the nodes in the clusters.

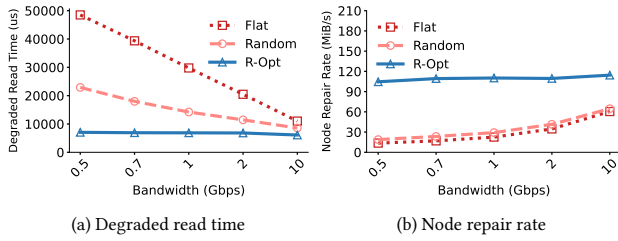


Figure 11: Exp#3: Repair performance under different cross-cluster bandwidths.

Discussion. A major limitation in our evaluation is that no user workload is fed. However, as our designed data placement minimizes the cross-cluster repair traffic, we believe that our data placement still notably outperforms the baseline flat and random data placements under real workloads. We pose this as our future work.

7 RELATED WORK

LRC designs. Theoretical studies on LRC provide explicit constructions [12, 13, 35], analyze the theoretical repair costs [8, 29], and explore the Minimum Hamming Distance [34, 35]. In terms of system implementation, LRC is implemented and evaluated in Azure [13], Facebook [32], and Ceph [16]. Recent studies [41, 42] analyze the optimal trade-offs between repair and scaling in Azure-LRC, and further design data placements that achieve the optimal trade-offs. The study [10] explores wide stripe erasure coding based on Azure-LRC, and designs data placement with minimized degraded read cost. Some studies [10, 21] improve the update performance of LRC. In contrast, our work mainly focuses on the optimal repair performance of a particular LRC construction, Optimal-LRC.

Optimal-LRC. Several studies on Optimal-LRC prove the optimality of the distance [8, 15], and design explicit constructions [2, 19, 34–36]. The study [16] implements Optimal-LRC in Ceph, and compares the fault tolerance and repair costs of Optimal-LRC with other LRC constructions. Our work is the first one that addresses the repair performance of Optimal-LRC in clustered storage systems.

Erasure coding in clustered storage. Recent studies focus on the deployment of erasure coding in clustered storage systems. Some studies design cluster-aware repair algorithms to minimize the cross-cluster repair bandwidth [9–11, 17, 26, 33], while the studies [41, 42] minimize the cross-cluster bandwidth for scaling. While previous studies mainly focus on Reed-Solomon Codes and Azure-LRC, we consider Optimal-LRC and present a repair-optimal data placement design.

8 CONCLUSION

We study how to optimize the repair performance of Optimal-LRC in clustered storage systems. We show that the data placement of Optimal-LRC has a great impact on the cross-cluster repair traffic. We explore the fault tolerance of Optimal-LRC, and then design an optimal data placement scheme that provably minimizes the cross-cluster repair traffic subject to single-cluster fault tolerance. We implement a key-value store prototype atop Memcached that

enables the optimal data placement scheme of Optimal-LRC. Both numerical studies and testbed experiments validate the efficiency of the optimal data placement scheme.

ACKNOWLEDGMENTS

This work is supported by National Natural Science Foundation of China (61832011, 62172382, 62141216). The corresponding author is Si Wu.

REFERENCES

- [1] Faraz Ahmad, Srimat T. Chakradhar, Anand Raghunathan, and T. N. Vijaykumar. 2014. ShuffleWatcher: Shuffle-aware Scheduling in Multi-tenant MapReduce Clusters. In *Proc. of USENIX ATC*.
- [2] Han Cai, Ying Miao, Moshe Schwartz, and Xiaohu Tang. 2020. On optimal Locally Repairable Codes with Super-Linear length. *IEEE Trans. on Information Theory* 66, 8 (2020), 4853–4868.
- [3] Haibo Chen, Heng Zhang, Mingkai Dong, Zhaoguo Wang, Yubin Xia, Haibing Guan, and Binyu Zang. 2017. Efficient and available in-memory kv-store with hybrid erasure coding and replication. *ACM Trans. on Storage (TOS)* 13, 3 (2017), 25.
- [4] Mosharaf Chowdhury, Srikanth Kandula, and Ion Stoica. 2013. Leveraging end-point flexibility in data-intensive clusters. In *Proc. of ACM SIGCOMM*.
- [5] cisco. 2015. Cisco Systems. Oversubscription and density best practices. https://www.cisco.com/c/en/us/solutions/collateral/data-centervirtualization/storage-networking/solution/net_implementation_white_paper0900aecd800f592f.html.
- [6] A. Fikes. 2010. Storage architecture and challenges. https://cloud.google.com/files/storage_architecture_and_challenges.pdf.
- [7] Daniel Ford, François Labelle, Florentina Popovici, Murray Stokely, Van-Anh Truong, Luiz Barroso, Carrie Grimes, and Sean Quinlan. 2010. Availability in globally distributed storage systems. In *Proc. of USENIX OSDI*.
- [8] Parikshit Gopalan, Cheng Huang, Huseyin Simitci, and Sergey Yekhanin. 2012. On the locality of codeword symbols. *IEEE Trans. on Information Theory* 58, 10 (2012), 6925–6934.
- [9] Hanxu Hou, Patrick P. C. Lee, Kenneth W. Shum, and Yuchong Hu. 2019. Rack-aware regenerating codes for data centers. *IEEE Trans. on Information Theory* 65, 8 (2019), 4730–4745.
- [10] Yuchong Hu, Liangfeng Cheng, Qiaori Yao, Patrick P. C. Lee, Weichun Wang, and Wei Chen. 2021. Exploiting combined locality for Wide-Stripe Erasure Coding in distributed storage. In *Proc. of USENIX FAST*.
- [11] Yuchong Hu, Xiaolu Li, Mi Zhang, Patrick P. C. Lee, Xiaoyang Zhang, Pan Zhou, and Dan Feng. 2017. Optimal repair layering for erasure-coded data centers: from theory to practice. *ACM Trans. on Storage* 13, 4 (2017), 24.
- [12] Cheng Huang, Minghua Chen, and Jin Li. 2013. Pyramid Codes: flexible schemes to trade space for access efficiency in reliable data storage systems. *ACM Trans. on Storage* 9, 1 (2013), 1–28.
- [13] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. 2012. Erasure coding in Windows Azure Storage. In *Proc. of USENIX ATC*.
- [14] Bert Hubert, Jacco Geul, and Simon Séhler. 2017. The wonder shaper 1.4. <https://github.com/magnific0/wondershaper>.
- [15] Oleg Kolosov, Alexander Barg, Itzhak Tamo, and Gala Yadgar. 2018. Optimal LRC codes for all lengths $n \leq q$. *CoRR abs/1802.00157* (2018).
- [16] Oleg Kolosov, Gala Yadgar, Matan Liram, Itzhak Tamo, and Alexander Barg. 2020. On fault tolerance, locality, and optimality in Locally Repairable Codes. *ACM Trans. on Storage* 16, 18 (2020), 865–877.
- [17] Xiaolu Li, Zuoru Yang, Jinhong Li, Runhui Li, Patrick P. C. Lee, Qun Huang, and Yuchong Hu. 2021. Repair pipelining for erasure-coded storage: algorithms and evaluation. *IEEE Trans. on Storage* 17, 2 (2021), 29.
- [18] libmemcached. 2014. Libmemcached. <https://libmemcached.org/libMemcached.html>.
- [19] Jian Liu, Sihem Mesnager, and Lusheng Chen. 2018. New constructions of optimal Locally Recoverable Codes via good polynomials. *IEEE Trans. on Information Theory* 64, 2 (2018), 889–899.
- [20] M.G. Luby, M. Mitzenmacher, M.A. Shokrollahi, and D.A. Spielman. 2001. Efficient erasure correcting codes. *IEEE Trans. on Information Theory* 47, 2 (2001), 569–584.
- [21] Mehrtaash Mehrabi, Mostafa Shahabinejad, Masoud Ardakani, and Majid Khabbazi. 2018. Improving the update complexity of Locally Repairable Codes. *IEEE Trans. on Communications* 66, 9 (2018), 3711–3720.
- [22] Memcached. 2021. Memcached. <https://memcached.org>.
- [23] Subramanian Muralidhar, Wyatt Lloyd, Sabyasachi Roy, Cory Hill, Ernest Lin, Weiwen Liu, Satadru Pan, Shiva Shankar, Viswanath Sivakumar, Linpeng Tang, and Sanjeev Kumar. 2014. F4: Facebook’s Warm BLOB Storage System. In *Proc. of USENIX OSDI*.

- [24] James S Plank, Jianqiang Luo, Catherine D Schuman, Lihao Xu, Zooko Wilcox-O'Hearn, et al. 2009. A performance evaluation and examination of open-source erasure coding libraries for storage. In *Proc. of USENIX FAST*.
- [25] James S Plank, Scott Simmerman, and Catherine D Schuman. 2008. *Jerasure: A library in C/C++ facilitating erasure coding for storage applications-version 1.2*. Technical Report. University of Tennessee, Tech. Rep. CS-08-627.
- [26] N. Prakash, Vitaly Abdrashitov, and Muriel Médard. 2018. The storage versus repair-bandwidth trade-off for clustered storage systems. *IEEE Trans. on Information Theory* 64, 8 (2018), 5783–5805.
- [27] KV Rashmi, Mosharaf Chowdhury, Jack Kosaian, Ion Stoica, and Kannan Ramchandran. 2016. EC-Cache: Load-Balanced, low-latency cluster caching with online erasure coding. In *Proc. of USENIX OSDI*.
- [28] K. V. Rashmi, Nihar B. Shah, Dikang Gu, Hairong Kuang, Dhruba Borthakur, and Kannan Ramchandran. 2013. A solution to the network challenges of data recovery in erasure-coded distributed storage systems: A study on the Facebook warehouse cluster. In *Proc. of USENIX HotStorage*.
- [29] Ankit Singh Rawat, Dimitris S. Papailiopoulos, Alexandros G. Dimakis, and Sriram Vishwanath. 2016. Locality and availability in distributed storage. *IEEE Trans. on Information Theory* 62, 8 (2016), 4481–4493.
- [30] Irving Reed and Gustave Solomon. 1960. Polynomial codes over certain finite fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304.
- [31] Sharad S. Sane. 2013. *Pigeonhole principle*. Hindustan Book Agency, 169–192.
- [32] Maheswaran Sathiamoorthy, Megasthenis Asteris, Dimitris Papailiopoulos, Alexandros G. Dimakis, Ramkumar Vadali, Scott Chen, and Dhruba Borthakur. 2013. XORing Elephants: novel Erasure Codes for big data. In *Proc. of VLDB Endowment*.
- [33] Zhirong Shen, Patrick P. C. Lee, Jiwu Shu, and Wenzhong Guo. 2020. Cross-rack-aware single failure recovery for clustered file systems. *IEEE Trans. on Dependable and Secure Computing* 17, 2 (2020), 248–261.
- [34] Natalia Silberstein, Ankit Singh Rawat, O. Ozan Koyluoglu, and Sriram Vishwanath. 2013. Optimal Locally Repairable Codes via rank-metric codes. In *Proc. of IEEE ISIT*.
- [35] Itzhak Tamo and Alexander Barg. 2014. A family of Optimal Locally Recoverable Codes. *IEEE Trans. on Information Theory* 60, 8 (2014), 4661–4676.
- [36] Itzhak Tamo, Dimitris S. Papailiopoulos, and Alexandros G. Dimakis. 2016. Optimal Locally Repairable Codes and connections to Matroid Theory. *IEEE Trans. on Information Theory* 62, 12 (2016), 6661–6671.
- [37] Ashish Vulimiri, Carlo Curino, P. Brighten Godfrey, Thomas Jungblut, Jitu Padhye, and George Varghese. 2015. Global analytics in the face of bandwidth and regulatory constraints. In *Proc. of USENIX NSDI*.
- [38] Hakim Weatherspoon and John D Kubiawicz. 2002. Erasure coding vs. replication: A quantitative comparison. In *Proc. of IPTPS*.
- [39] Shuzhan Wei, Yongkun Li, Yinlong Xu, and Si Wu. 2017. DSC: Dynamic stripe construction for synchronous encoding in clustered file system. In *Proc. of IEEE INFOCOM*.
- [40] Si Wu, Qingpeng Du, Patrick P. C. Lee, Yongkun Li, and Yinlong Xu. 2022. Optimal data placement for stripe merging in Locally Repairable Codes. In *Proc. of IEEE INFOCOM*.
- [41] Si Wu, Zhirong Shen, and Patrick PC Lee. 2020. On the optimal repair-scaling trade-off in Locally Repairable Codes. In *Proc. of IEEE INFOCOM*.
- [42] Si Wu, Zhirong Shen, Patrick P. C. Lee, and Yinlong Xu. 2022. Optimal repair-scaling trade-off in Locally Repairable Codes: analysis and evaluation. *IEEE Trans. on Parallel and Distributed Systems* 33, 1 (2022), 56–69.
- [43] Mingyuan Xia, Mohit Saxena, Mario Blaum, and David A Pease. 2015. A tale of two erasure codes in HDFS. In *Proc. of USENIX FAST*.
- [44] Qiaori Yao, Yuchong Hu, Liangfeng Cheng, Patrick PC Lee, Dan Feng, Weichun Wang, and Wei Chen. 2021. StripeMerge: Efficient wide-stripe generation for large-scale erasure-coded storage. In *Proc. of IEEE ICDCS*.